

Fundamentos de Organización de Datos

Clase 7

Agenda

Hashing

- Definición
- Tipos
- Propiedades

Propiedades

- Función de hash
- Densidad / tamaño nodo
- Tratamiento del overflow

Dispersión

- Estática
- Dinámica

Hashing (Dispersión) → Introducción

Necesitamos un mecanismo de acceso a registros con una lectura solamente

- Secuencial : $N/2$ accesos promedio
- Ordenado (búsqueda binaria) : $\log_2 N$
- Árboles : 3 o 4 accesos

Clave Primarias → características

- No se repiten
- El resto de las claves actúan a través de ella
- Cuando se aprenda a modelar, tendrán más características que las hacen especiales

Hashing (Dispersión) → Definición

Técnica para generar una dirección base única para una clave dada. La dispersión se usa cuando se requiere acceso rápido a una clave

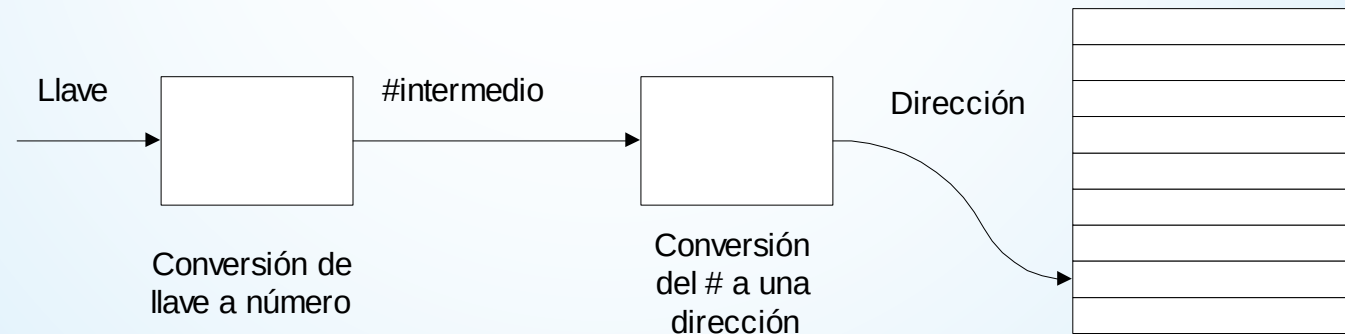
Técnica que convierte la clave del registro en un número aleatorio, el que sirve después para determinar donde se almacena el registro.

Técnica de almacenamiento y recuperación que usa una función de hash para mapear registros en direcciones de memoria secundaria.

Hashing (Dispersión) → Definición

Atributos del hash

- No requiere almacenamiento adicional (índice)
- Facilita inserción y eliminación rápida de registros
- Encuentra registros con muy pocos accesos al disco en promedio



Hashing (Dispersión) → Definición

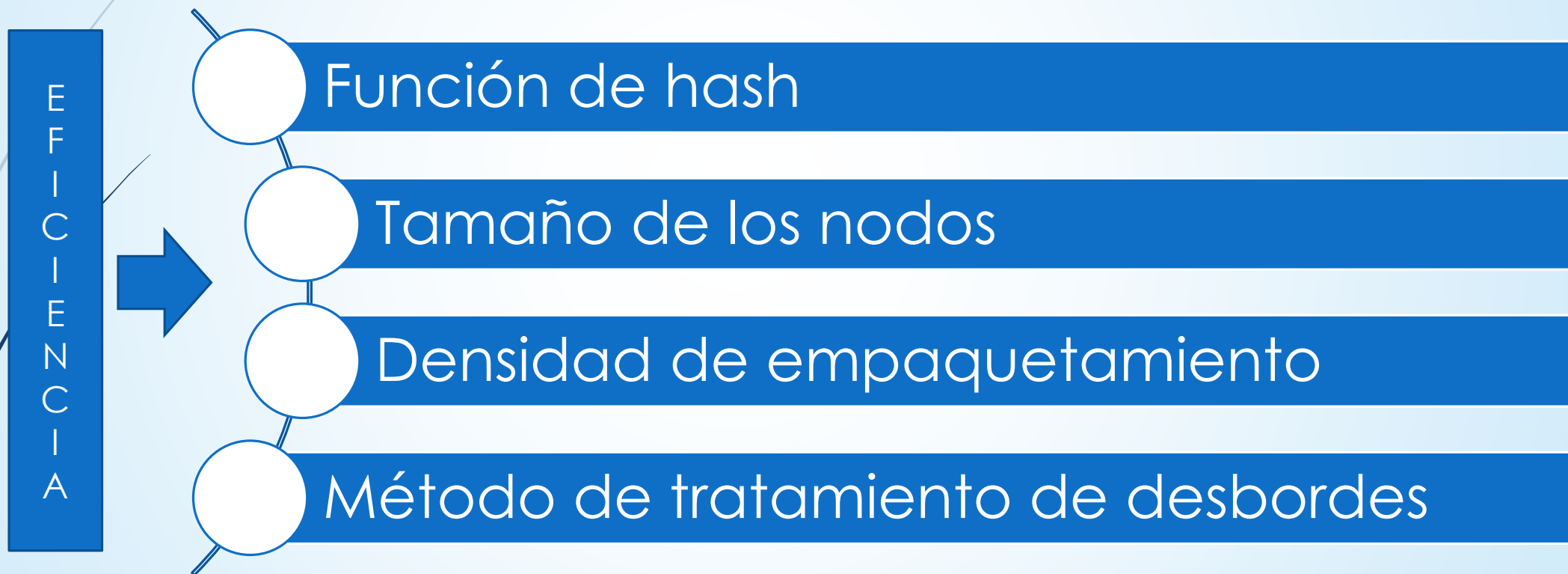
Costo

- No podemos usar registros de longitud variable
- No puede haber orden físico de datos
- No permite claves duplicadas

Para determinar la dirección

- La clave se convierte en un número casi aleatorio
- # se convierte en una dirección de memoria
- El registro de datos completo se guarda en esa dirección
- Si la dirección está ocupada → colisión y posible overflow (tratamiento especial)

Hashing (Dispersión) → Parámetros



Hashing (Dispersión) → Parámetros

1. Función de hash

- Caja negra que a partir de una clave se obtiene la dirección donde debe estar el registro.
- Diferencias con índices
 - Dispersión no hay relación aparente entre clave y dirección
 - Dos claves distintas pueden transformarse en iguales direcciones (claves sinónimos)

Hashing (Dispersión) → parámetros

Colisión:

- Situación en la que un registro es asignado a una dirección que está utilizada por otro registro.

Overflow

- Situación en la que un registro es asignado a una dirección en la cual no queda espacio para alojarlo.

Soluciones

- Algoritmos de dispersión sin colisiones o que estas colisiones nunca produzcan overflow → (perfectos) (imposibles de conseguir).
- Almacenar los registros de alguna otra forma, esparcir.

Hashing (Dispersión) → Parámetros

¹⁰ Soluciones para las colisiones

- Esparcir registros: buscar métodos que distribuyan los registros de la forma más aleatoria posible
- Usar memoria adicional: distribuir pocos registros en muchas direcciones:
 - Disminuye el colisiones y por ende disminuye el overflow
 - Desperdicia espacio
- Colocar más de un registro por dirección: direcciones con N claves → mejoras notables
 - Ej: archivo con registro físicos de 512 bytes y el registro a almacenar es de 80 bytes → se puede almacenar hasta 6 registros por cada dirección de archivo.
 - Cada dirección tolera hasta 5 sinónimos
 - Las direcciones que pueden almacenar varios registros en esta forma → **nodos/cubetas/compartimentos**

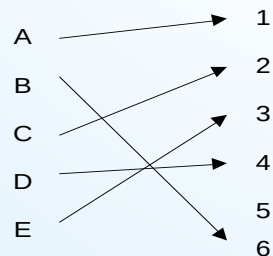
Hashing (Dispersión) → Parámetros

Algoritmos simples de dispersión

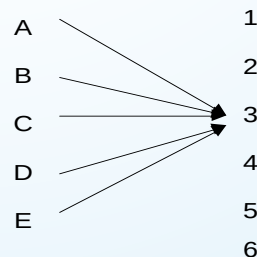
- Condiciones
 - Repartir registros en forma uniforme
 - Aleatoria (las claves son independientes, no influyen una sobre la otra)

Tres pasos

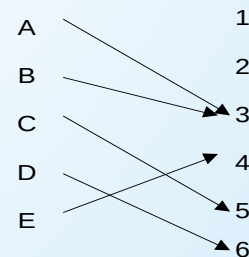
- Representar la llave en forma numérica (en caso que no lo sea)
- Aplicar la función
- Relacionar el número resultante con el espacio disponible



uniforme



peor



aceptable

Hashing (Dispersión) → Parámetros

■ Ejemplo de Funciones de dispersión

- **Centros cuadrados:** la llave se multiplica por si misma y tomando los dígitos centrales al cuadrado, posteriormente se ajusta al espacio disponible
- **División:** la clave se divide por un # aproximadamente igual al # de direcciones (número primo pues tiende a distribuir residuos en forma más eficiente)
- **Desplazamiento:** los dígitos externos de ambos extremos se corren hacia adentro, se suman y se ajusta al espacio disponible

Hashing (Dispersión) → Parámetros

2. Tamaño de los nodos/cubetas/compartimentos

- Puede almacenar más de un registro
- A mayor tamaño
 - Menor colisión
 - Mayor fragmentación
 - Búsqueda más lenta dentro de la cubeta (este concepto realmente afecta al problema?)

3. Densidad de empaquetamiento

- Proporción de espacio del archivo asignado que en realidad almacena registros
- $DE = \frac{\text{número de registros del archivo}}{\text{capacidad total del archivo}}$
- Densidad de empaquetamiento menor
 - Menos overflow
 - Más desperdicio de espacio

Hashing (Dispersión) → Parámetros

Estimación del overflow → sabiendo que

- N # de cubetas,
- C capacidad de nodo,
- R # reg. Del archivo
- $DE = \frac{R}{C \times N}$
- Probabilidad que una cubeta reciba I registros (distribución de Poisson)

$$P(I) = \frac{R!}{I! * (R - I)!} * \left(\frac{1}{N}\right)^I * \left(1 - \frac{1}{N}\right)^{R-I}$$

$$P(I) = \frac{(R/N)^I * e^{-(R/N)}}{I!}$$

Hashing (Dispersión) → Parámetros

Por que?Cuál es la justificación de la fórmula anterior?

- Supongamos que
 - A: no utilizar un cubeta particular
 - B: utilizar una cubeta en particular
- $P(B) = 1/N$ $P(A) = 1 - P(B) = 1 - 1/N$
- Si tenemos dos llaves?
 - $P(BB) = P(B) * P(B) = (1/N)^2$ (porque se puede asegurar esto?)
 - $P(BA) = P(B) * P(A) = (1/N) * (1 - 1/N)$
 - $P(AA) = P(A) * P(A) = (1 - 1/N)^2$

Hashing (Dispersión) → Parámetros

Si la secuencia fuera de tres claves

- $P(BBB) \circ P(BBA) \circ P(BAB)$
- Cuantas combinaciones? → 8

En general → si fueran R claves

- $P(A...AB...B)$ siendo la suma de A y B igual a R
- Que nos interesa → que l registros vayan a un nodo

Hashing (Dispersión) → Parámetros

$$P(I) = \frac{R!}{I! * (R - I)!} * \left(\frac{1}{N}\right)^I * \left(1 - \frac{1}{N}\right)^{R-I}$$

En general la secuencia de K llaves, que I caigan en un nodo es la probabilidad

$$\left(\frac{1}{N}\right)^I * \left(1 - \frac{1}{N}\right)^{R-I}$$

Cuántas formas de combinar esta probabilidad hay (R tomadas de a I combinaciones)

$$\frac{R!}{I! * (R - I)!}$$

Función de Poisson: (probabilidad que un nodo tenga I elementos) R, N, I con la definición ya vista

$$P(I) = \frac{(R/N)^I * e^{-(R/N)}}{I!}$$

Hashing (Dispersión) → Parámetros

Análisis numérico de Hashing

- En general si hay n direcciones, entonces el # esperado de direcciones con I registros asignados es $N * P(I)$.

- Las colisiones aumentan con el archivo más “lleno”

• Ej: $N = 10000$ $K = 10000$ $DE = 1$ 100%

$$P(I) = \frac{(R/N)^I * e^{-(R/N)}}{I!}$$

$$P(0) = 0.3679$$

$$3679$$

$$P(1) = 0.3679$$

*

$$10000$$

$$3679$$

qué significa?

$$P(2) = 0.1839$$

$$1839$$

$$P(3) = 0.0613$$

$$613$$

$$P(4), P(5), \dots$$

$$\text{Overflow o saturación} = 1839 + 2 * 613 = 3065 \quad (\text{alto})$$

Hashing (Dispersión) → Parámetros

20

Ahora supongamos que el problema es

- $K = 500$ $N = 1000$ $DE = 50\%$

$$P(0) = 0.607 \quad 607$$

$$P(1) = 0.303 \quad * 1000 \quad 303$$

$$\text{overflow o saturación} = N * [1 * P(2) + 2 * P(3) + 3 * P(4) + 4 * P(5)] = \mathbf{107}$$

- **Saturación menor**

densidad

10%

50%

100%

overflow

4.8%

21.4% ($107 / 500$) → (reg. saturación/k)

36.8%

- si la DE es del 50% y cada dirección puede almacenar sólo un registro, puede esperarse que aprox. el 21% de los registros serán almacenados en algún lugar que no sea sus direcciones base
- **los números bajos de overflow (baja densidad) → muchas cubetas libres**

Hashing (Dispersión) → Parámetros

Que pasa si mantenemos la DE pero cambiamos ciertos valores

- EJ:

$$\begin{array}{l} K = 750 \\ N = 1000 \\ C = 1 \end{array} \Rightarrow \begin{array}{l} DE = 75\% \\ K / N = 0.75 \end{array}$$

$$\begin{array}{l} K = 750 \\ N = 500 \\ C = 2 \end{array} \Rightarrow \begin{array}{l} DE = 75\% \\ K / N = 1.5 \end{array}$$

deben influir en la función de Poisson

saturación $c = 1 \longrightarrow 222$ cubetas
 $c = 2 \longrightarrow 140$ cubetas

- Cual es el tamaño de la cubeta?

Hashing (Dispersión) → Parámetros

DE	1	2	5	10	100
10%	4.8	0.6	0.0	0.0	0.0
20%	9.4	2.2	0.1	0.0	0.0
30%	13.6	4.5	0.4	0.0	0.0
40%	17.6	7.3	1.1	0.1	0.0
50%	21.4	10.4	2.5	0.4	0.0
60%	24.8	13.7	4.5	1.3	0.0
70%	28.1	17.0	7.1	2.9	0.0
75%	29.6	18.7	8.6	4.0	0.0
80%	31.2	20.4	10.3	5.3	0.1
90%	34.1	23.8	13.8	8.9	0.8
100%	36.8	27.1	17.6	12.5	4.0

Hashing (Dispersión) → Parámetros

Tratamiento de Colisiones con Overflow

- Hemos visto que el % de overflow se puede reducir, pero el problema se mantiene dado que no llegamos a 0%

Algunos métodos

- Saturación progresiva
- Saturación progresiva encadenada
- Doble dispersión
- Área de desborde separado

Hashing (Dispersión) → Parámetros

Saturación progresiva:

- Cuando se completa el nodo, se busca el próximo hasta encontrar uno libre.
- Búsqueda?
- Eliminación, no debe obstaculizar las búsquedas

Hashing (Dispersión) → Parámetros

saturación progresiva encadenada

- similar a saturación progresiva, pero los reg. de saturación se encadenan y “no ocupan” necesariamente posiciones contiguas
- Ejemplo

Hashing (Dispersión) → Parámetros

Dispersión doble:

- saturación tiende a agrupar en zonas contiguas, búsquedas largas cuando la densidad tiende a uno
- Solución almacenar los registros de overflow en zonas no relacionadas.
- esquema con el cual se resuelven overflows aplicando una segunda función a la llave para producir un N° C, el cual se suma a la dirección original tantas veces como sea necesario hasta encontrar una dirección con espacio.

Hashing (Dispersión) → Parámetros

Encadenamiento en áreas separadas:

- No utiliza nodos de direcciones para los overflow, estos van a nodos especiales
- Ejemplo:
- Se mejora el tratamiento de inserciones o eliminaciones. Empeora el TAP.
- Ubicación del desborde
 - A intervalos regulares entre direcciones asignadas
 - Cilindros de desborde

Hashing (Dispersión)

Hash con espacio de direccionamiento estático

- Necesita un número de direcciones fijas, virtualmente imposible
- Cuando el archivo se llena
 - Saturación excesiva
 - Redispersar, nueva función, muchos cambios

Solución → espacio de direccionamiento dinámico

- Reorganizar tablas sin mover muchos registros
- Técnicas que asumen bloques físicos, pueden utilizarse o liberarse.

Hashing (Dispersión) → espacio dinámico

Varias posibilidades

- Hash virtual
- Hash dinámico
- Hash Extensible

Hash Extensible

- Adapta el resultado de la función de hash de acuerdo al número de registros que tenga el archivo, y de las cubetas necesitadas para su almacenamiento.
- Función: Genera secuencia de bits (normalmente 32)

Hashing (Dispersión) → espacio dinámico

- El principio del método consiste en comenzar a trabajar con un único nodo para almacenar registros e ir aumentando la cantidad de direcciones disponibles a medida que los nodos se completan.
- Al ser dinámico, **no utiliza el concepto de DE**
- El método necesita, para su implementación, de una **estructura auxiliar**. Esta estructura es una tabla que se administra en memoria principal, aunque se almacene en memoria secundaria
- La función de hash retorna un **string de bits**. La cantidad de bits que retorna determina la cantidad máxima de direcciones a las que puede acceder el método.
- Dicha secuencia permite obtener de la tabla en memoria la dirección física del nodo para almacenar el registro. Además, la tabla será utilizada posteriormente para recuperar cada registro en **un solo acceso a disco**.
- Si la función de hash retorna 32 bits, en ese caso es posible direccionar 2^{32} direcciones de nodos diferentes, si fuera necesario.
- Se utilizan solo los bits necesarios de acuerdo a cada instancia del archivo.

Hashing (Dispersión) → espacio dinámico

- Los bits tomados forman la dirección del nodo que se utilizará
- Si se intenta insertar un registro en una cubeta o nodo lleno, deben reubicarse todos los registros allí contenidos entre el nodo viejo y el nuevo, y para esto se toma un bit más.
- La tabla tendrá tantas entradas (direcciones de nodos) como 2^i , siendo i el número de bits actuales para el sistema.
- **El tratamiento de dispersión con la política de *hash* extensible comienza con un solo nodo en disco, y una tabla que solamente contiene una dirección, la del único nodo disponible.**
- **Inicialmente, el número cero sobre la tabla indica que no es necesario ningún bit de la secuencia obtenida por la función de dispersión, para obtener la dirección física donde almacenar el registro.**
- **Además, inicialmente, el nodo en disco también tiene cero.**

Hashing (Dispersión) → espacio dinámico (ejemplo)

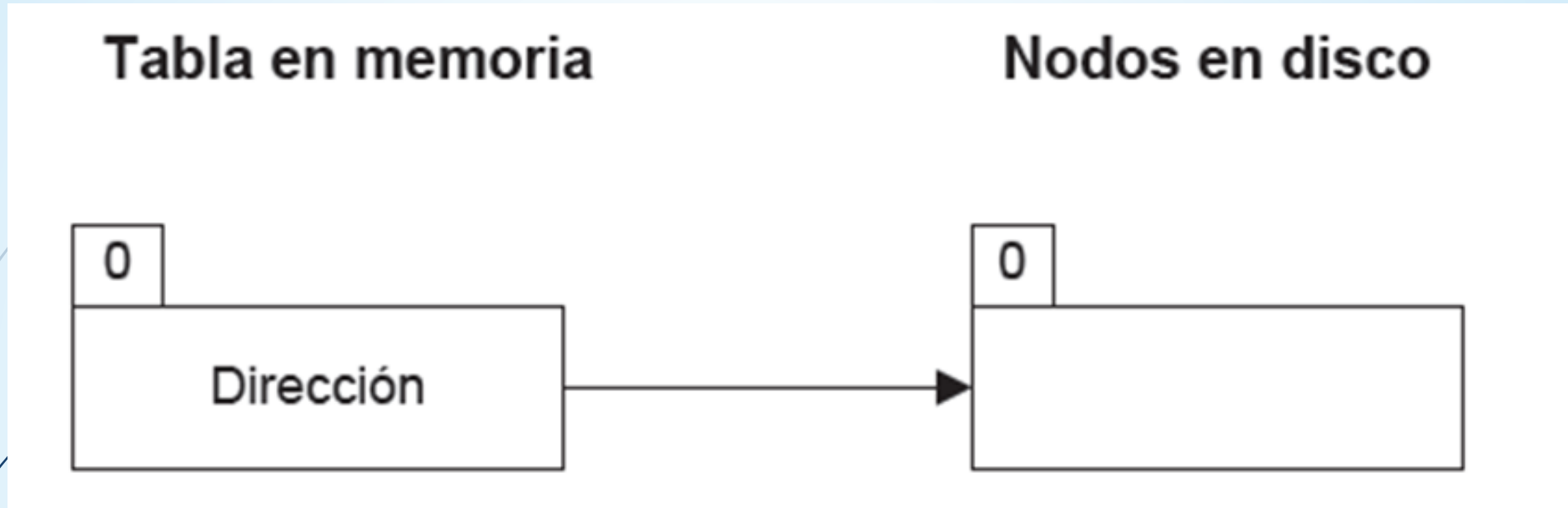
32

Clave	FH(Clave)
Alfa	00.....1001
Beta	00.....0100
Gamma	00.....0010
Delta	00.....1111
Epsilon	00.....0000
Rho	00.....1011
Pi	00.....0110
Tau	00.....1101
Psi	00.....0001
Omega	00.....0111

La función de hash retorna un string de bits. La cantidad de bits que retorna determina la cantidad máxima de direcciones a las que puede acceder el método.

Hashing (Dispersión) → espacio dinámico

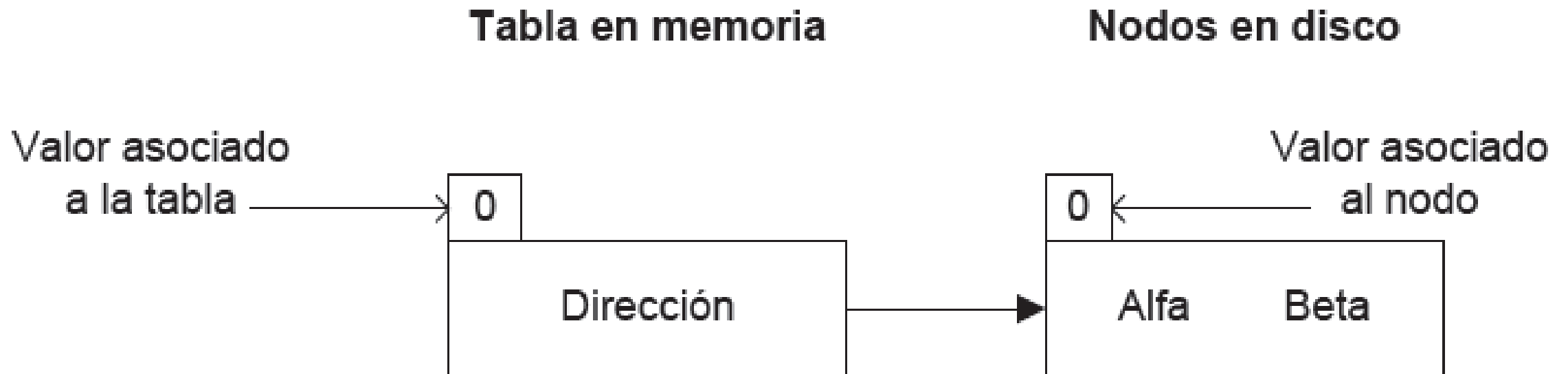
33



El tratamiento de dispersión con la política de hash extensible comienza con un solo nodo en disco, y una tabla que solamente contiene una dirección, la del único nodo disponible.

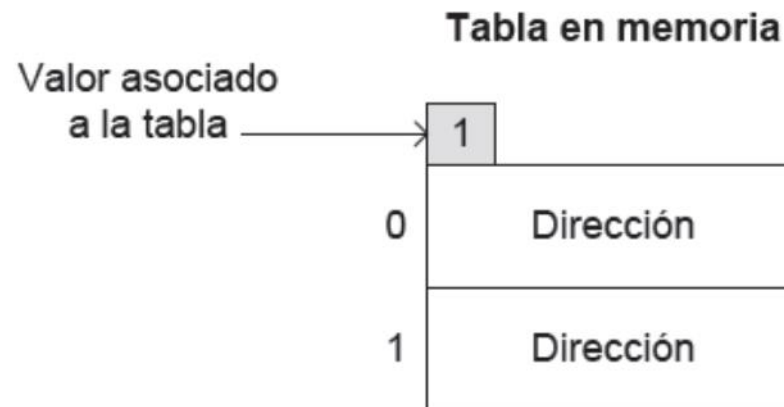
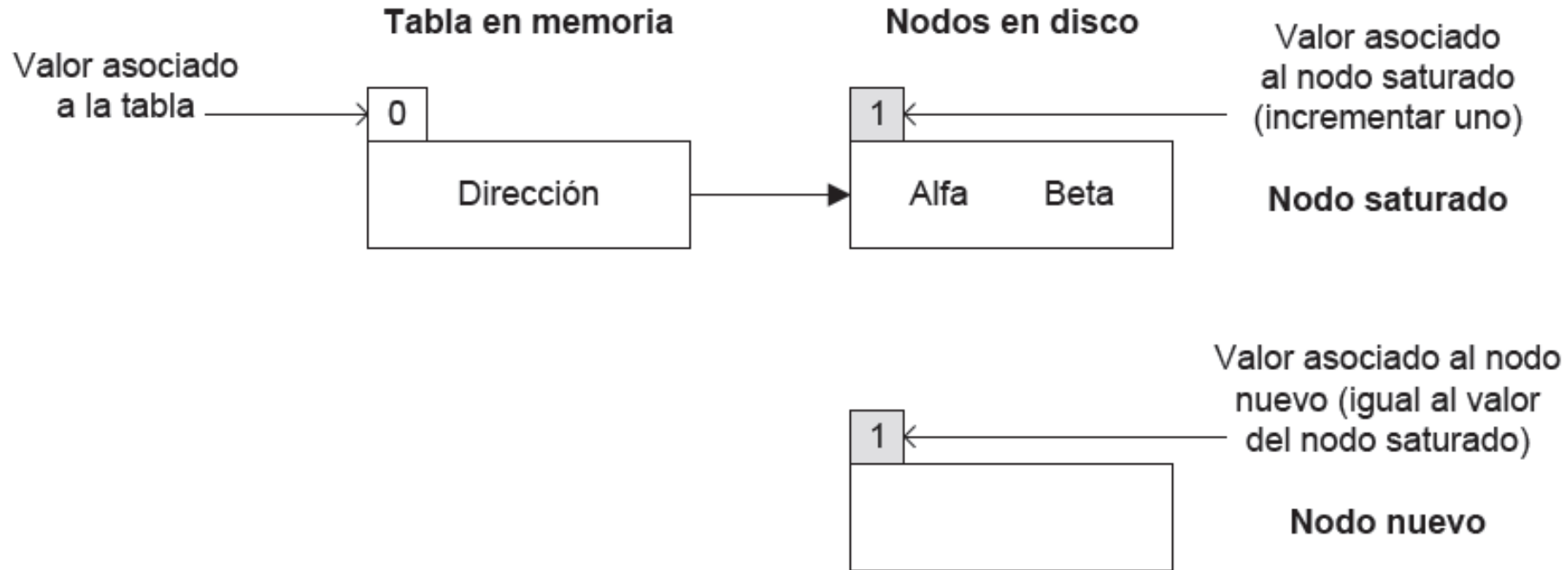
Inicialmente, el número cero sobre la tabla indica que no es necesario ningún bit de la secuencia obtenida por la función de dispersión, para obtener la dirección física donde almacenar el registro.

Hashing (Dispersión) → espacio dinámico

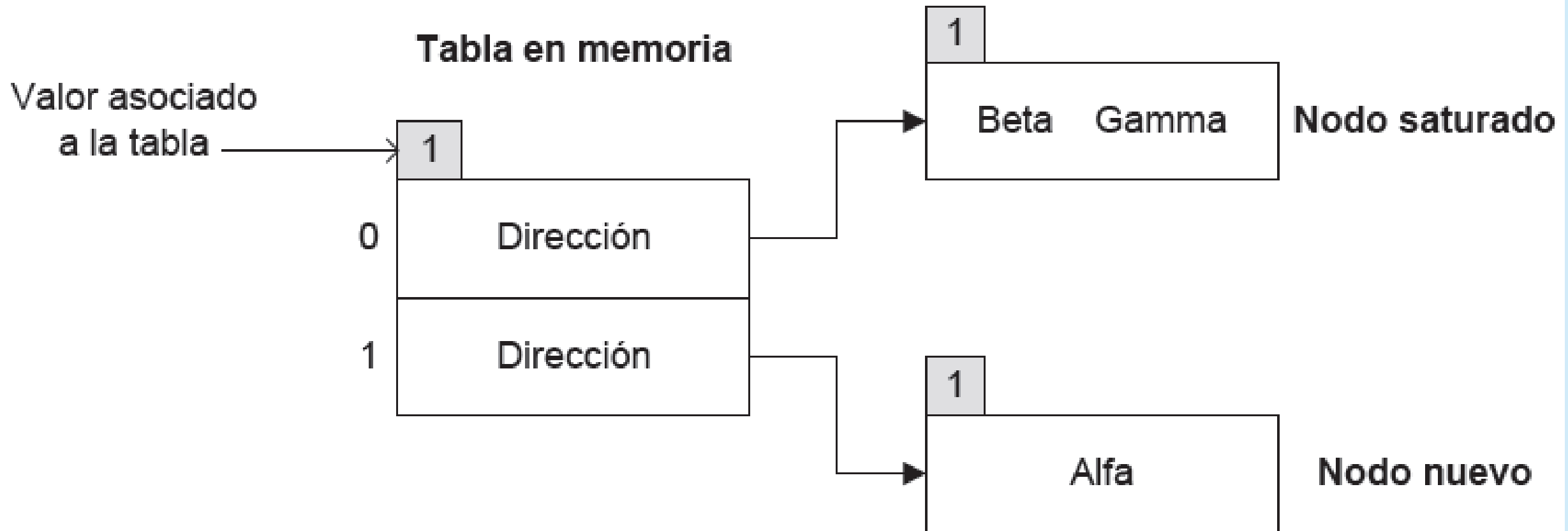


Hashing (Dispersión) → espacio dinámico

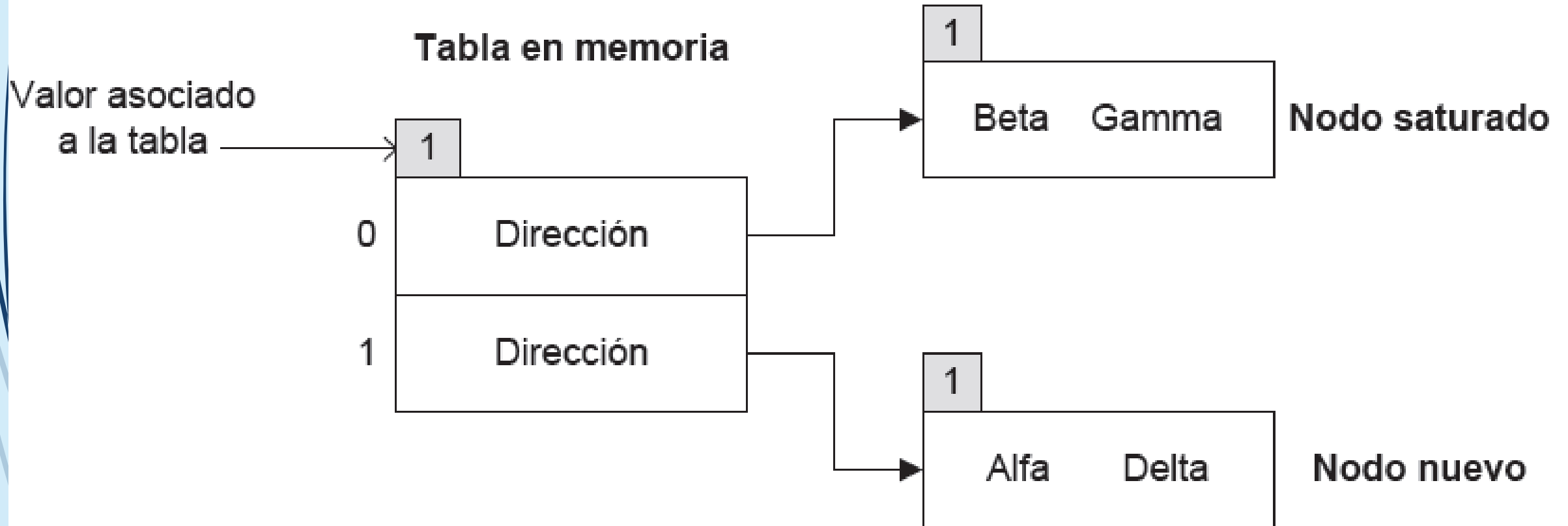
35



Hashing (Dispersión) → espacio dinámico



Hashing (Dispersión) → espacio dinámico



Hashing (Dispersión) → espacio dinámico

38

Nodos en disco

Valor asociado
al nodo saturado
(incrementar uno)

Nodo saturado

Valor asociado al nodo
nuevo (igual al valor
del nodo saturado)

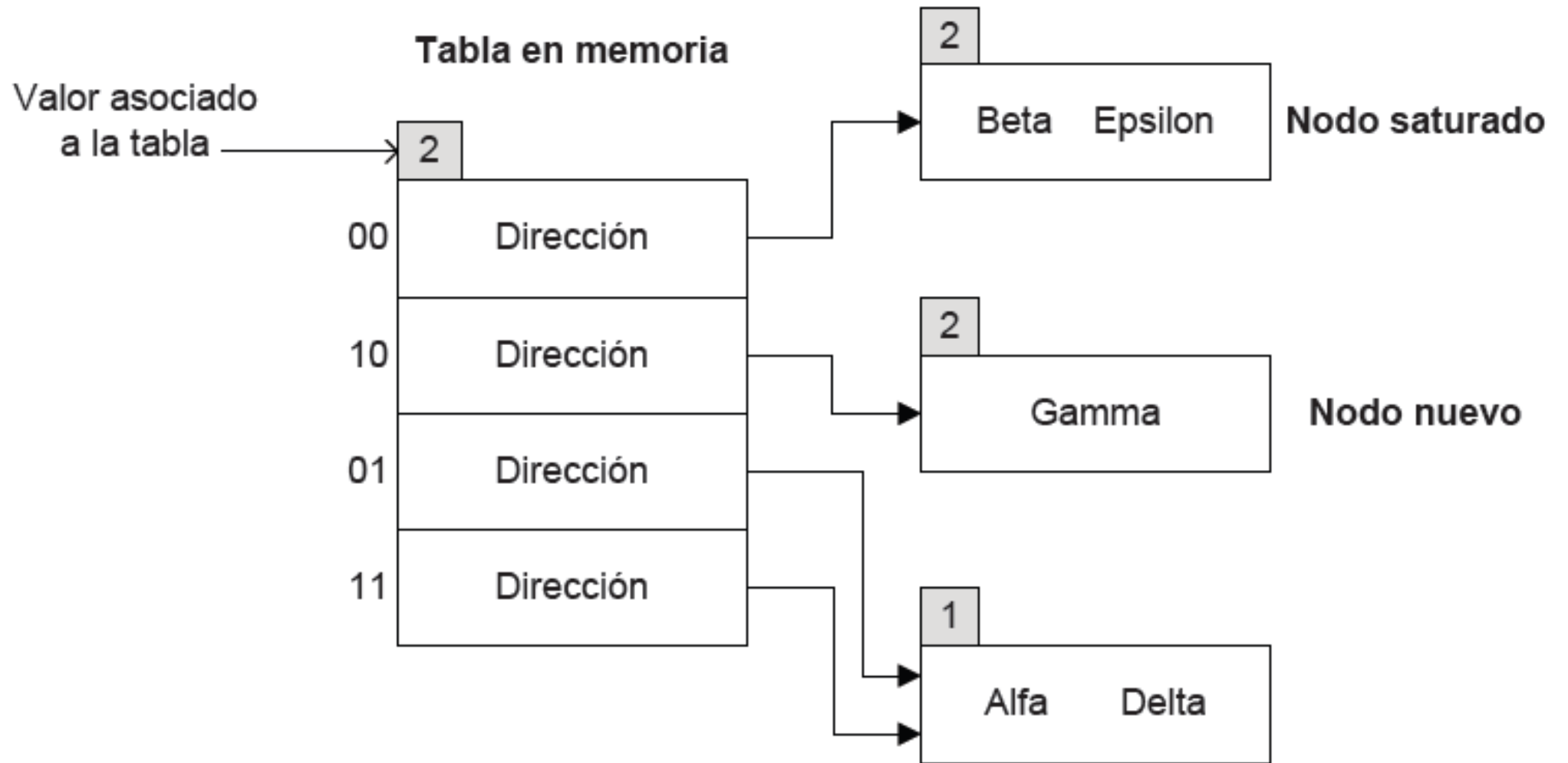
Nodo nuevo

Tabla en memoria

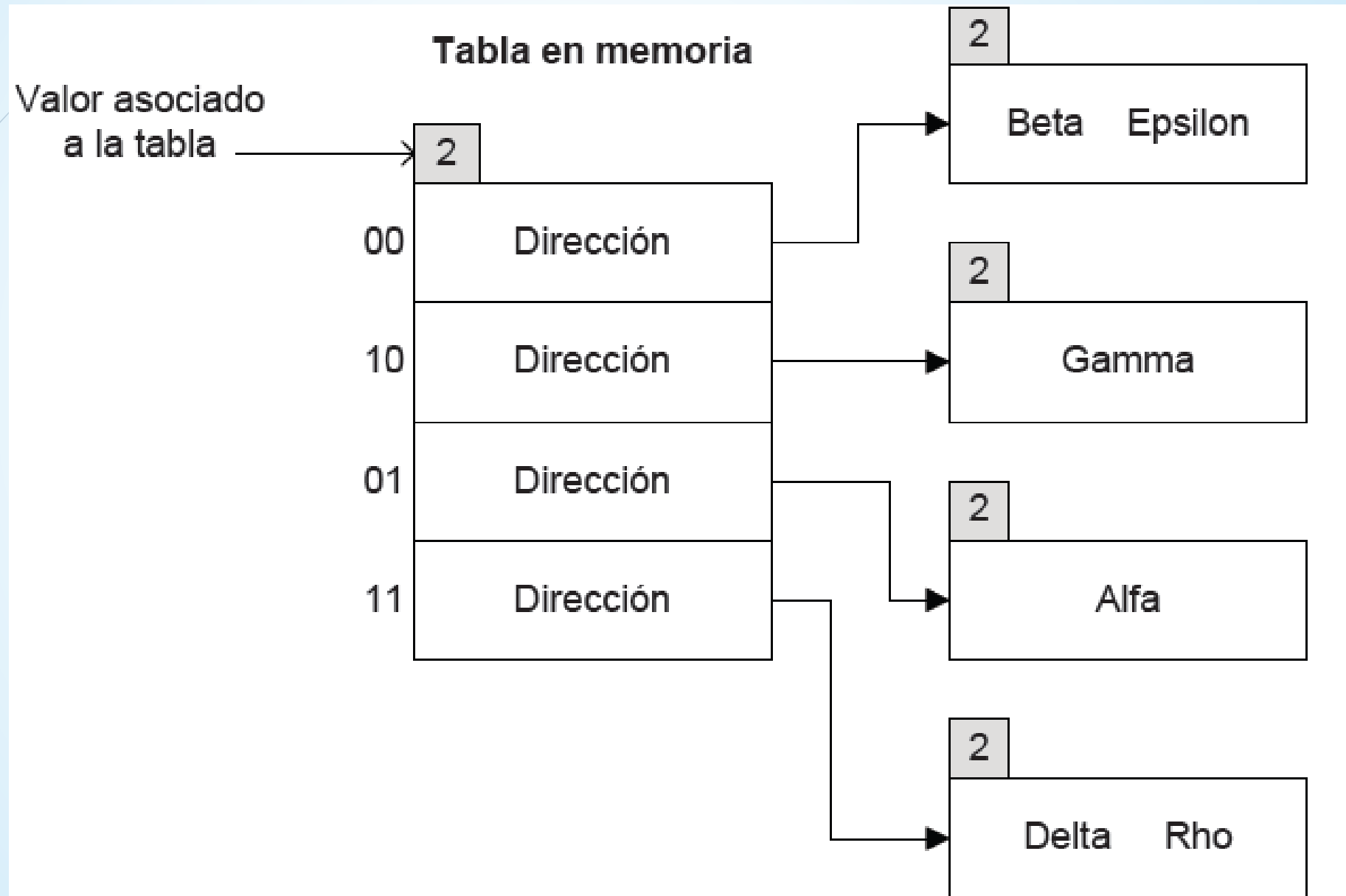
Valor asociado
a la tabla

2	
00	Dirección
10	Dirección
01	Dirección
11	Dirección

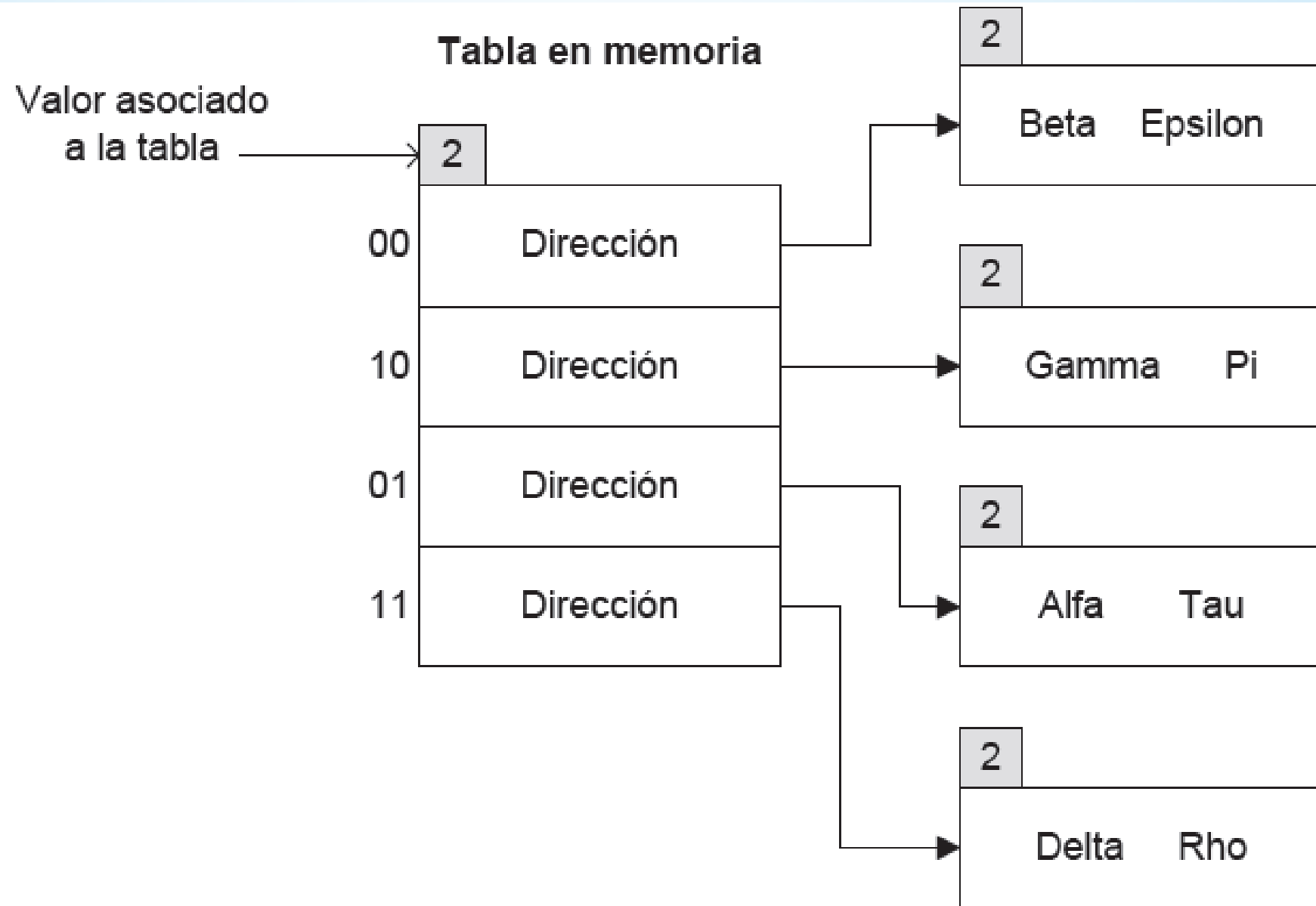
Hashing (Dispersión) → espacio dinámico



Hashing (Dispersión) → espacio dinámico



Hashing (Dispersión) → espacio dinámico



Hashing (Dispersión) → espacio dinámico

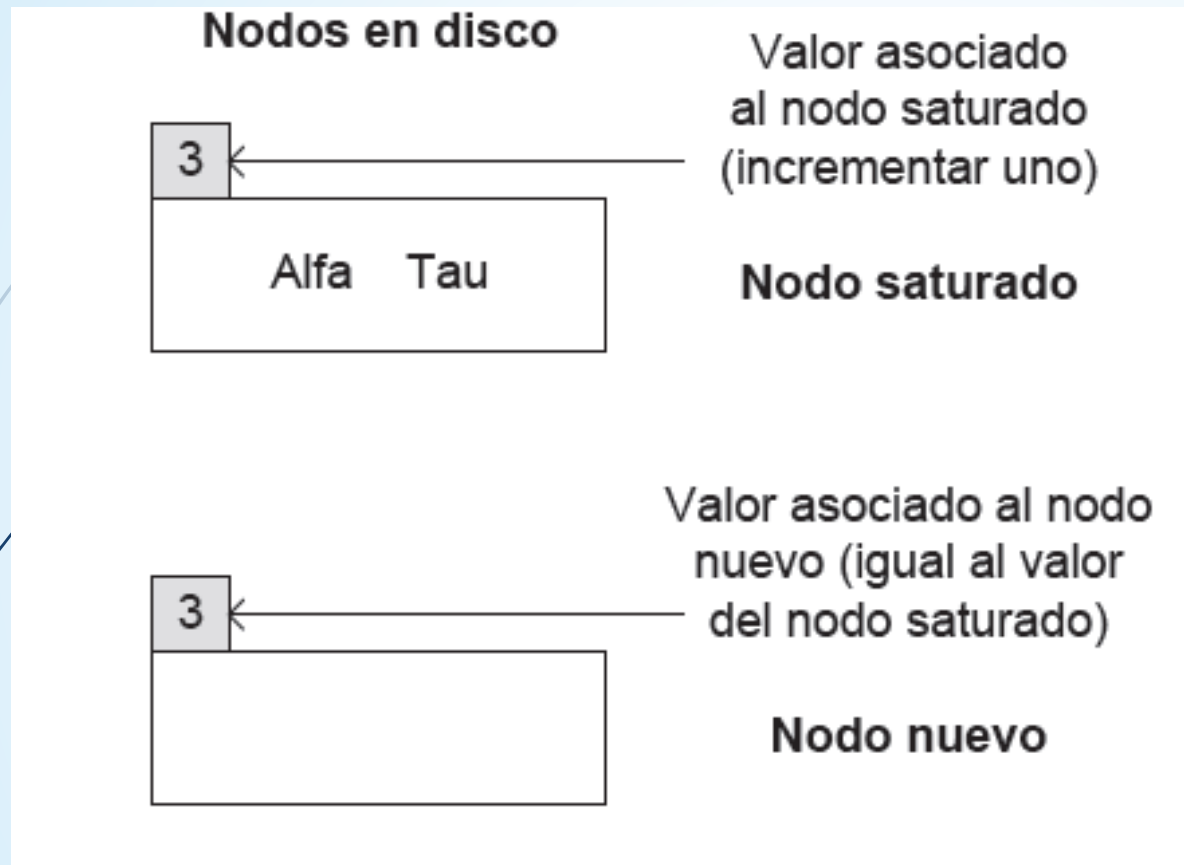
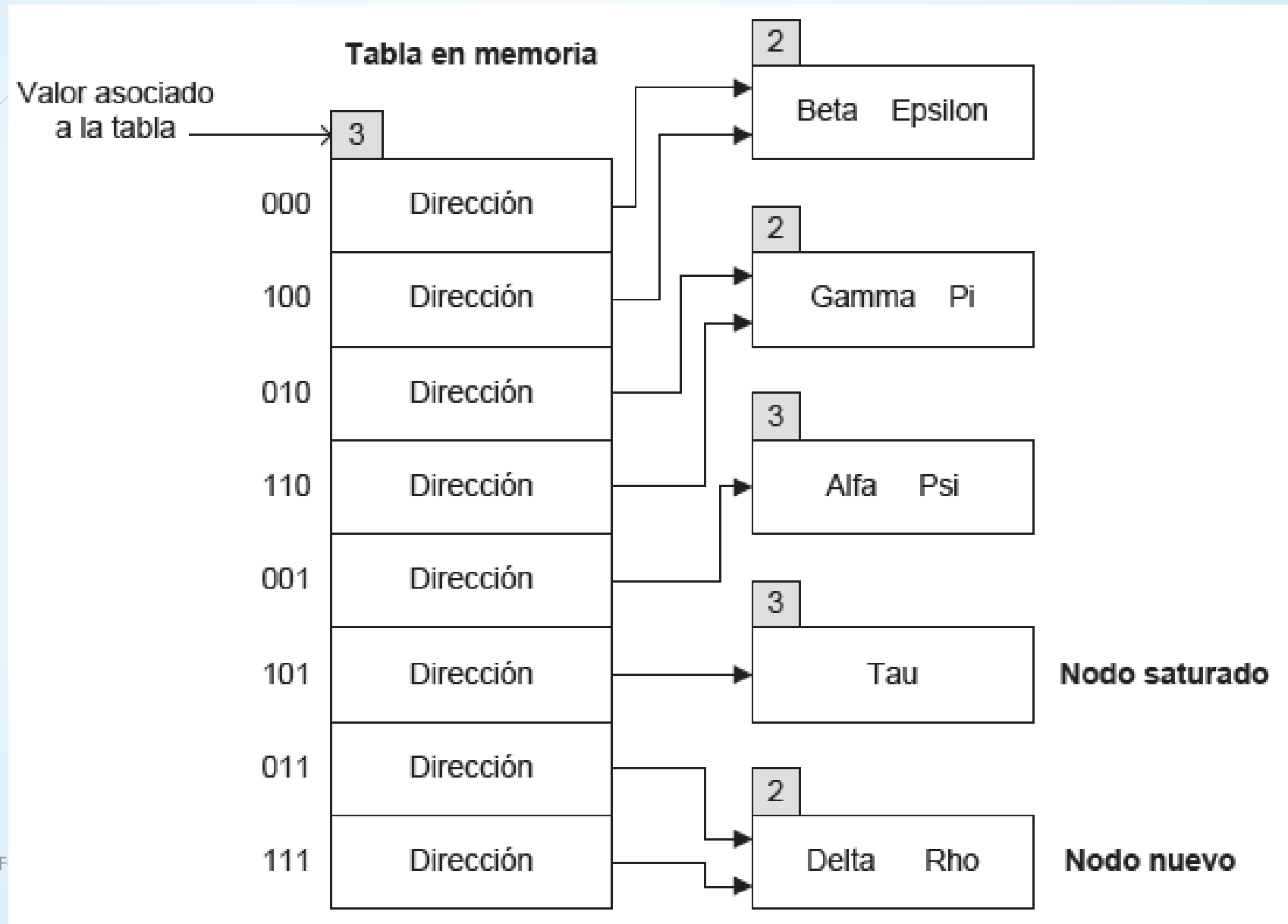


Tabla en memoria

Valor asociado a la tabla → 3

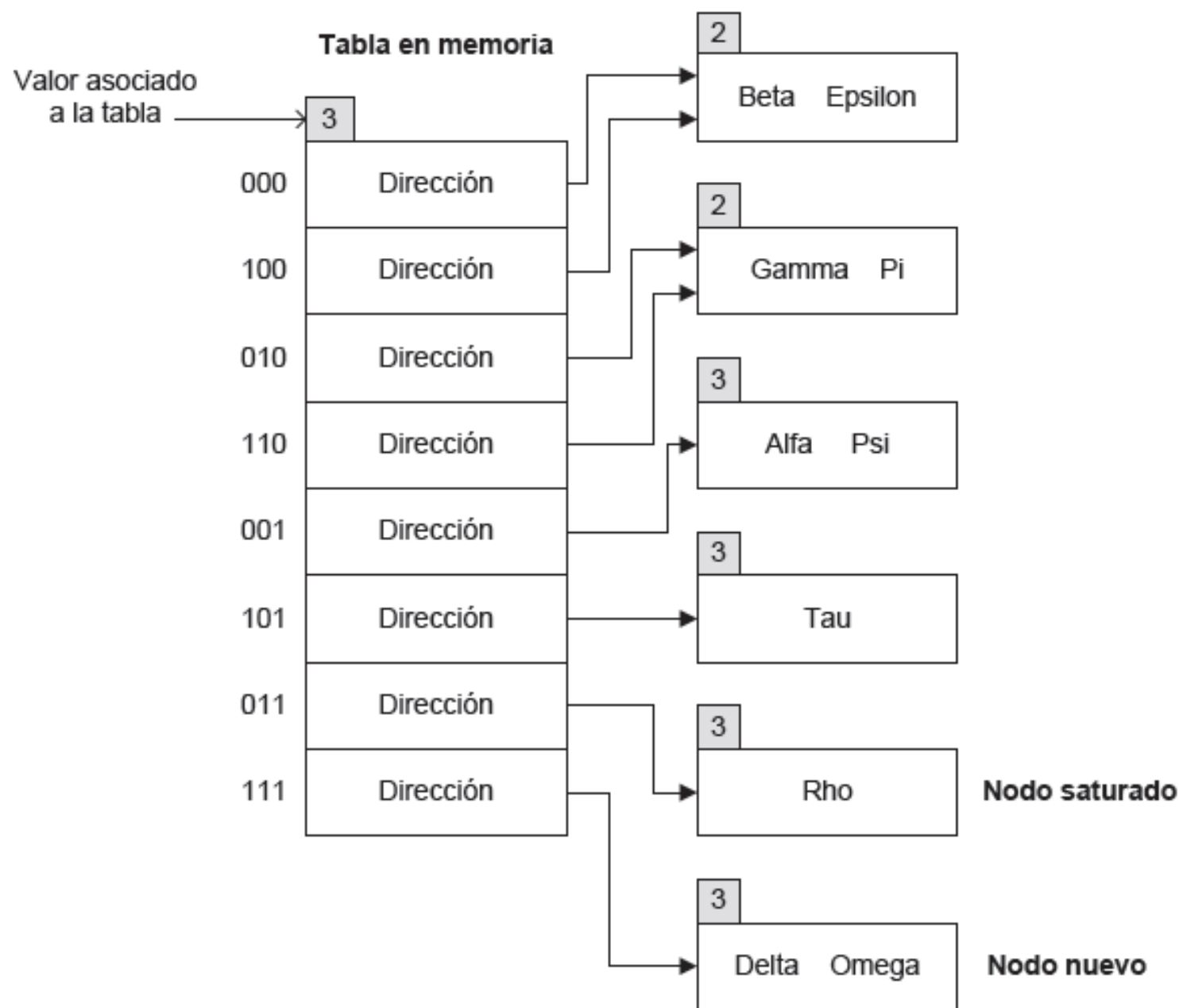
000	Dirección
100	Dirección
010	Dirección
110	Dirección
001	Dirección
101	Dirección
011	Dirección
111	Dirección

Hashing (Dispersión) → espacio dinámico



Hashing (Dispersión) → espacio dinámico

44



Hashing (Dispersión) → espacio dinámico

Resumiendo:

- Se utilizan solo los bits necesarios de acuerdo con cada instancia del archivo.
- Los bits tomados forman la dirección del nodo que se utilizará.
- **Si se intenta insertar en un nodo lleno, deben reubicarse todos los registros allí contenidos entre el nodo viejo y el nuevo; para ello se toma 1 bit más.**
- La tabla tendrá tantas entradas (direcciones de nodos) como 2^i , siendo i el número de bits actuales para el sistema.
- **El proceso de búsqueda asegura encontrar cada registro en un solo acceso.**
- Se calcula la secuencia de bits para la clave, se toman tantos bits de esa clave como indique el valor asociado a la tabla, y la dirección del nodo contenida en la celda respectiva debería contener el registro buscado.
- En caso de no encontrar el registro en dicho nodo, el elemento no forma parte del archivo de datos.

Elección de organización

Archivos

- Acomodar datos para satisfacer rápidamente requerimientos
- Accesos: resumen

Organización	Acc.un reg. CP	Todos reg. CP
Ninguna	Lento	Lento
Secuencial	Lento	Rápido
Index sec.	Buena	Rápida
Hash	Rápido	lento

Elección de organización

Elección de organización

- Captar los **requerimientos** de usuario
- Que examinar
 - Características del archivo
 - Número de registros, tamaño de registros
 - Requerimientos de usuario
 - Tipos de operaciones, número de accesos a archivos
 - Características del hard
 - Tamaño de sectores, bloques, pistas, cilindros, etc.
 - Parámetros
 - Tiempo (necesario para desarrollar y mantener el soft, para procesar archivos)
 - Uso promedio (# reg. Usados/ #registros)